



PABNA UNIVERSITY OF SCIENCE AND TECHNOLOGY

Faculty of Engineering and Technology

Department of Information and Communication Engineering
(ICE)

PRACTICAL LAB REPORT

COURSE INFORMATION

Course Code: ICE-4204

Course Title: System Analysis and Software Testing Sessional

SUBMITTED BY

Md. Nazmul Hossain Emon

Roll No.: 200621

Session: 2019-20

Year / Sem: 4th Year, 2nd Semester

Department: Information &
Communication Engineering

University: Pabna University of Science
and Technology

SUBMITTED TO

Dr. Md. Anwar Hossain

Designation: Professor

Department: Information &
Communication Engineering

University: Pabna University of Science
and Technology

Location: Pabna, Bangladesh

Date of Submission:

Pb. No	Problem Name	Page No.
01	Write a program in "JAVA" or "C" to develop a simple calculator that would be able to take a number, an operator (addition/subtraction/multiplication/ division/modulo) and another number consecutively as input and the program will display the output after pressing "=" sign. Sample input: 1+2; 8%4; Sample output: 1+2=3; 8%4=0.	01
02	Write a program in "JAVA" or "C" that will take two 'n' integers as input until a particular operator and produce 'n' output. Sample input: 4 5 7 8 20 40 +; Sample output: 9 15 60.	03
03	Write a program in "JAVA" or "C" to check whether a number or string is palindrome or not. N.B: your program must not take any test case number such as 1 or 2 for the desired cases from the user. Program user will insert a number or string as input directly and the program will display the exact result in the output console.	05
04	Write a program in "JAVA" or "C" to find out the factorial of a number using a while and for loop. Also verify the results obtained from each case.	07
05	Write a program in "JAVA" or "C" that will find the sum and average of an array using do-while loop and 2 user-defined functions.	09
06	Write a simple "JAVA" program to explain ClassNotFoundException and EOFException exception.	11
07	Write a program in "JAVA" or "C" that will read a input.txt file containing n positive integers and calculate addition, subtraction, multiplication and division in separate output.txt file. Sample input: 5 5 9 8; Sample output: Case-1:10 0 25 1; Case-2: 17 1 72 1	14
08	Write Down the ATM system specifications and report the various bugs.	17
09	Explain the role of software engineering in Biomedical Engineering and in the field of Artificial Intelligence and Robotics.	18
10	Study the various phases of Water-fall model. Which phase is the most dominated one?	20
11	Using COCOMO model estimate effort for specific problem in industrial domain	22
12	Identify the reasons behind software crisis and explain the possible solutions for the following scenario: Case 1: "Air ticket reservation software was delivered to the customer and was installed in an airport 12.00 AM (mid night) as per the plan. The system worked quite fine till the next day 12.00 PM (noon). The system crashed at 12.00 PM and the airport authorities could not continue using software for ticket reservation till 5.00 PM. It took 5 hours to fix the defect in the software". Case 2: "Software for financial systems was delivered to the customer. Customer informed the development team about a mal-function in the system. As the software was huge and complex, the development team could not identify the defect in the software".	24

Problem No: 01

Problem Name:

Write a program in “JAVA” or “C” to develop a simple calculator that would be able to take a number, an operator (addition/subtraction/multiplication/division/modulo) and another number consecutively as input and the program will display the output after pressing the “=” sign.

Sample input: 8-1; 9+2; 8%4;

Sample output: 8-1=7; 9+2=11; 8%4=0.

Objectives:

1. To design and implement a simple calculator program in C that performs arithmetic operations.
2. To accept two numbers and an operator as input and compute the corresponding result.
3. To demonstrate the use of conditional statements (switch-case) in C programming.

Theory:

A calculator is one of the fundamental software applications that performs basic arithmetic operations such as addition, subtraction, multiplication, division, and modulo. In programming, a simple calculator can be implemented using conditional constructs.

In C, the switch-case statement is particularly well-suited for menu-driven or operator-driven programs, as it allows selecting a block of code based on the value of a variable or expression. The scanf() function is used to read formatted input, and printf() is used to produce formatted output.

Key Concepts:

Arithmetic Operators: +, -, *, /, % — each performs a defined mathematical computation.

Input/Output in C: scanf() reads values from standard input; printf() writes formatted values to standard output.

Switch-Case: Evaluates an expression and executes the matching case block, making the code readable and efficient.

Division by Zero: A special check must be incorporated to handle division and modulo operations where the divisor is zero, preventing runtime errors.

The program flow is: read the first operand → read the operator → read the second operand → compute result → display output.

Algorithm:

Step 1: Start the program.

Step 2: Declare variables: two floating-point numbers (num1, num2) and a character variable (op) for the operator.

Step 3: Prompt the user to enter the first number, the operator, and the second number.

Step 4: Read the inputs using scanf().

Step 5: Use a switch-case statement to evaluate the operator.

Step 6: For '+', compute num1 + num2. For '-', compute num1 – num2. For '*', compute num1 × num2.

Step 7: For '/', check if num2 is zero; if so, display 'Division by zero error'; otherwise compute num1 / num2.

Step 8: For '%', check if num2 is zero; if so, display an error; otherwise compute (int)num1 % (int)num2.

Step 9: For the default case, display 'Invalid operator'.

Step 10: Display the result in the format: num1 op num2 = result. Stop the program

Source Code:

```
#include <stdio.h>

int main() {
    double num1, num2;
    char op;

    /* Read: number, operator, number */
    printf("Enter expression (e.g., 8-1): ");
    scanf("%lf %c %lf", &num1, &op, &num2);

    printf("%.0lf %c %.0lf = ", num1, op, num2);

    switch (op) {
        case '+': printf("%.2lf\n", num1 + num2); break;
        case '-': printf("%.2lf\n", num1 - num2); break;
        case '*': printf("%.2lf\n", num1 * num2); break;
        case '/':
            if (num2 == 0) printf("Error: Division by zero\n");
            else printf("%.2lf\n", num1 / num2);
            break;
        case '%':
            if ((int)num2 == 0) printf("Error: Division by zero\n");
            else printf("%d\n", (int)num1 % (int)num2);
            break;
        default: printf("Invalid operator\n");
    }
    return 0;
}
```

Sample Input:

```
Enter expression (e.g., 8-1): 8 - 1
Enter expression (e.g., 8-1): 9 + 2
Enter expression (e.g., 8-1): 8 % 4
```

Sample Output:

```
8 - 1 = 7.00
9 + 2 = 11.00
8 % 4 = 0
```

Problem No: 02

Problem Name:

Write a program in “JAVA” or “C” that will take two ‘n’ integers as input until a particular operator and produce ‘n’ output.

Sample input: 4 5 7 8 20 40 +

Sample output: 9 15 60

Objectives:

1. To write a program that reads pairs of integers and a single operator, then applies the operator to each pair.
2. To demonstrate the use of arrays and loops for batch processing of input data.
3. To understand how a single operator can be applied consistently across multiple data pairs.

Theory:

This problem requires reading an even number of integers followed by an arithmetic operator. The integers are then paired (1st with 2nd, 3rd with 4th, ...) and the specified operation is applied to each pair.

Arrays in C:

An array is a data structure that stores a collection of elements of the same type in contiguous memory locations. Arrays allow processing of multiple inputs using loop constructs, significantly reducing code redundancy.

Operator-Driven Batch Processing:

Rather than asking for an operator per operation, a single operator is collected at the end and applied uniformly. This approach is common in bulk data processing and command-line utility design.

The algorithm reads integers until it encounters a non-numeric token (the operator), stores them in an array, and then iterates through pairs, computing the result for each.

Loop Constructs:

A for loop is used to iterate over array indices. Pairing is achieved by incrementing the index by 2 per iteration, processing elements at positions i and $i+1$ simultaneously.

Algorithm:

- Step 1:** Start the program.
- Step 2:** Declare an integer array of sufficient size and a character variable for the operator.
- Step 3:** Use scanf() in a loop to read integers into the array until the operator character is encountered.
- Step 4:** Count the total number of integers read (n).
- Step 5:** Read the operator character from input.
- Step 6:** Initialise a loop index $i = 0$ and iterate while $i < n$, incrementing i by 2 at each step.
- Step 7:** Inside the loop, apply the operator to $arr[i]$ and $arr[i+1]$ using a switch-case.
- Step 8:** Print each computed result separated by a space.
- Step 9:** After all pairs are processed, print a newline and stop.

Source Code:

```
#include <stdio.h>

int main() {
    int arr[100];
    int n = 0;
    char op;
    char token[20];
    printf("Enter integers followed by an operator: ");
    /* Read tokens; stop when a non-integer is found */
    while (scanf("%d", &arr[n]) == 1) {
        n++;
    }
    scanf(" %c", &op); /* read operator */
    printf("Results: ");
    for (int i = 0; i + 1 < n; i += 2) {
        int result = 0;
        switch (op) {
            case '+': result = arr[i] + arr[i+1]; break;
            case '-': result = arr[i] - arr[i+1]; break;
            case '*': result = arr[i] * arr[i+1]; break;
            case '/':
                if (arr[i+1] != 0) result = arr[i] / arr[i+1];
                break;
            case '%': result = arr[i] % arr[i+1]; break;
        }
        printf("%d ", result);
    }
    printf("\n");
    return 0;
}
```

Sample Input

4 5 7 8 20 40 +

Sample Output

Results: 9 15 60

Problem No: 03

Problem Name:

Write a program in “JAVA” or “C” to check whether a number or string is a palindrome or not.

N.B.: Your program must not take any test case number such as 1 or 2 for the desired cases from the user. The program user will insert a number or string as input directly, and the program will display the exact result in the output console.

Objectives:

1. To develop a C program that checks whether an input number or string is a palindrome or not.
2. To understand string manipulation and character comparison techniques in C programming.
3. To learn how conditional statements and loops are used to compare characters from both ends of a string.

Theory:

A palindrome is a sequence of characters (or digits) that reads the same forward and backward. Classic examples include 'madam', 'racecar', '121', and '12321'.

Palindrome Check for Strings:

The standard approach compares characters from both ends moving inward. If every pair of characters matches, the string is a palindrome. The algorithm terminates early if a mismatch is found.

Palindrome Check for Numbers:

A number can be checked by reversing its digits (obtained via modulo and integer division) and comparing the reversed number to the original. Alternatively, the number is converted to a string and the string palindrome algorithm is applied.

String Functions in C:

strlen() returns the length of a string. strcpy() copies one string to another. Manual reversal uses a two-pointer technique: one pointer at the start and one at the end, swapping characters until they meet.

Since the problem forbids accepting a test-case number, the program accepts input, detects whether it is numeric or alphabetic, and applies the appropriate check automatically.

Algorithm

- Step 1:** Start the program.
- Step 2:** Declare a character array (str) of adequate size.
- Step 3:** Prompt the user to enter a number or a string without specifying the type.
- Step 4:** Read the input using scanf() into str.
- Step 5:** Calculate the length L of str using strlen().
- Step 6:** Initialise two pointers: left = 0 and right = L – 1.
- Step 7:** Use a while loop: while left < right, compare str[left] and str[right].
- Step 8:** If any pair of characters differs, set a flag isPalindrome = false and break.
- Step 9:** Increment left and decrement right after each successful comparison.
- Step 10:** If the flag remains true, print '<input> is a Palindrome'; otherwise print '<input> is NOT a Palindrome'. Stop.

Source Code

```
#include <stdio.h>
#include <string.h>
#include <stdbool.h>

int main() {
    char str[200];

    printf("Enter a number or string: ");
    scanf("%s", str);

    int len = strlen(str);
    int left = 0;
    int right = len - 1;
    bool isPalindrome = true;

    while (left < right) {
        if (str[left] != str[right]) {
            isPalindrome = false;
            break;
        }
        left++;
        right--;
    }

    if (isPalindrome)
        printf("\"%s\" is a Palindrome.\n", str);
    else
        printf("\"%s\" is NOT a Palindrome.\n", str);

    return 0;
}
```

Sample Input

```
Enter a number or string: madam
Enter a number or string: 12321
Enter a number or string: hello
```

Sample Output

```
"madam" is a Palindrome.
"12321" is a Palindrome.
"hello" is NOT a Palindrome
```

Problem No: 04

Problem Name:

Write a program in “JAVA” or “C” to find out the factorial of a number using a while and for loop. Also, verify the results obtained from each case.

Objectives:

1. To develop a C program that calculates the factorial of a given number using both for loop and while loop.
2. To understand iterative approaches for solving mathematical problems in programming.
3. To compare the results of both loop structures and verify their correctness.

Theory

The factorial of a non-negative integer n , denoted $n!$, is the product of all positive integers less than or equal to n . By definition, $0! = 1$.

Mathematically: $n! = n \times (n-1) \times (n-2) \times \dots \times 2 \times 1$

Factorials grow extremely rapidly; for example, $10! = 3,628,800$ and $20!$ exceeds the capacity of a 32-bit integer, necessitating the use of long long or unsigned long long in C.

For Loop:

The for loop is a count-controlled loop that initialises a counter, checks a condition before each iteration, and updates the counter after each iteration. It is ideal when the number of iterations is known in advance.

While Loop:

The while loop is a condition-controlled loop that checks its condition before each iteration. It is more general and useful when the termination condition is complex.

Both loops, when computing factorial, give identical results. The equivalence of while and for loops is an important concept in structured programming.

Algorithm

Step 1: Start the program.

Step 2: Declare a long long integer variable n for input and two long long variables $\text{factFor} = 1$ and $\text{factWhile} = 1$ to store results.

Step 3: Prompt the user and read the integer n .

Step 4: If $n < 0$, display 'Factorial undefined for negative numbers' and stop.

Step 5: Compute factorial using a for loop: `for (i = 1; i <= n; i++) factFor *= i.`

Step 6: Compute factorial using a while loop: `i = 1; while (i <= n) { factWhile *= i; i++; }`

Step 7: Display the result from the for loop: 'For loop result: $\langle n \rangle! = \langle \text{factFor} \rangle$ '.

Step 8: Display the result from the while loop: 'While loop result: $\langle n \rangle! = \langle \text{factWhile} \rangle$ '.

Step 9: Verify results: if $\text{factFor} == \text{factWhile}$, print 'Both results match!'; otherwise print 'Mismatch detected!'. And then stop the program.

Source Code

```
#include <stdio.h>

int main() {
    long long n, i;
    long long factFor = 1;
    long long factWhile = 1;
    printf("Enter a non-negative integer: ");
    scanf("%lld", &n);
    if (n < 0) {
        printf("Factorial is undefined for negative numbers.\n");
        return 1;
    }
    /* --- Using for loop --- */
    for (i = 1; i <= n; i++) {
        factFor *= i;
    }
    /* --- Using while loop --- */
    i = 1;
    while (i <= n) {
        factWhile *= i;
        i++;
    }
    printf("For loop result: %lld! = %lld\n", n, factFor);
    printf("While loop result: %lld! = %lld\n", n, factWhile);
    if (factFor == factWhile)
        printf("Verification: Both results match!\n");
    else
        printf("Verification: Mismatch detected!\n");
    return 0;
}
```

Sample Input

Enter a non-negative integer: 7

Sample Output

For loop result: 7! = 5040

While loop result: 7! = 5040

Problem No: 05

Problem Name:

Write a program in “JAVA” or “C” that will find the sum and average of an array using do-while loop and 2 user-defined functions.

Objectives:

To develop a C program that calculates the sum and average of array elements using a do-while loop and two user-defined functions.

Theory

An array is a contiguous block of memory that stores multiple values of the same data type. Computing the sum and average of array elements is a fundamental operation encountered in statistical analysis, grade computation, and signal processing.

Do-While Loop:

The do-while loop guarantees at least one iteration because the condition is evaluated after the loop body executes. This is particularly useful for input validation and menu-driven programs where the body must execute at least once.

User-Defined Functions:

Functions promote code reusability and modularity. In C, a function is defined with a return type, a name, and a parameter list. The computeSum() function returns the total of all array elements, while computeAverage() divides the sum by the number of elements and returns a floating-point result.

Average:

The arithmetic mean (average) of n values is defined as: $\text{Average} = \text{Sum} / n$. Care must be taken to cast at least one operand to float or double to avoid integer division truncation.

Algorithm

- Step 1:** Start the program.
- Step 2:** Declare integer array arr[], integer n (array size), and floating-point variables sum and avg.
- Step 3:** Prompt the user to enter the number of elements n.
- Step 4:** Use a do-while loop to read n integers into arr[].
- Step 5:** Call the user-defined function computeSum(arr, n) to calculate and return the sum of all elements.
- Step 6:** Call the user-defined function computeAverage(sum, n) to calculate and return the average.
- Step 7:** Display the original array elements.
- Step 8:** Display the computed sum: 'Sum = <value>'.
- Step 9:** Display the computed average: 'Average = <value>'.
- Step 10:** Stop the program.

Source Code

```
#include <stdio.h>

/* User-defined function: computes sum */
long long computeSum(int arr[], int n) {
    long long sum = 0;
    int i = 0;
    do {
        sum += arr[i];
```

```

        i++;
    } while (i < n);
    return sum;
}

/* User-defined function: computes average */
double computeAverage(long long sum, int n) {
    return (double)sum / n;
}

int main() {
    int arr[100], n, i = 0;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter %d integers:\n", n);
    do {
        scanf("%d", &arr[i]);
        i++;
    } while (i < n);

    long long sum = computeSum(arr, n);
    double avg = computeAverage(sum, n);

    printf("Array : ");
    for (int j = 0; j < n; j++) printf("%d ", arr[j]);
    printf("\n");
    printf("Sum : %lld\n", sum);
    printf("Average : %.2lf\n", avg);

    return 0;
}

```

Sample Input

```

Enter number of elements: 5
Enter 5 integers:
10 20 30 40 50

```

Sample Output

```

Array: 10 20 30 40 50
Sum: 150
Average: 30.00

```

Problem No: 06**Problem Name:**

Write a simple “JAVA” program to explain the classNotFound Exception and endOfFile(EOF) exception.

Objectives:

1. To understand the concept of exception handling in Java programming.
2. To demonstrate how ClassNotFoundException occurs during dynamic class loading.
3. To explain how EOFException occurs during file input operations and how it can be handled properly.

Theory:

Exception handling is an important feature in Java that allows programmers to detect and manage runtime errors without terminating the program unexpectedly. Exceptions improve the reliability and stability of software systems.

1. ClassNotFoundException

ClassNotFoundException is a checked exception in Java. It occurs when the Java Virtual Machine (JVM) tries to load a class dynamically at runtime using methods like Class.forName(), but the specified class cannot be found.

Causes:

- Incorrect class name
- Missing .class file
- Class not included in the classpath

Importance:

Handling this exception is necessary in applications that use:

- Dynamic class loading
- Reflection
- Plugin systems

2. EOFException (End Of File Exception)

EOFException is a subclass of IOException. It occurs when a program tries to read data beyond the end of a file or input stream.

Causes:

- Attempting to read more data than available in the file
- Unexpected end of file during input operation

Importance:

Handling EOFException helps programs terminate file-reading operations safely and prevents unexpected crashes.

This experiment helps to understand:

- Java exception handling
- Try-catch blocks
- File handling
- Runtime error management
- Dynamic class loading

Algorithm:**For ClassNotFoundException:**

1. Start
2. Use Class.forName() to load a non-existing class
3. If the class is not found, catch the exception
4. Display the exception message

5. End

For EOFException:

1. Open a file using FileInputStream and DataInputStream
2. Read integers continuously from the file
3. When the file ends unexpectedly, EOFException occurs
4. Catch the exception and display a message
5. End

Source Code

```
import java.io.*;

public class ExceptionDemo {

    /* ---- 1. ClassNotFoundException Demo ---- */
    static void demoClassNotFound() {
        System.out.println("=== ClassNotFoundException Demo ===");
        try {
            // Attempt to load a class that does not exist
            Class.forName("com.nonexistent.DatabaseDriver");
        } catch (ClassNotFoundException e) {
            System.out.println("ClassNotFoundException caught!");
            System.out.println("Message : " + e.getMessage());
        }
    }

    /* ---- 2. EOFException Demo ---- */
    static void demoEOF() {
        String filename = "data.bin";
        System.out.println("\n=== EOFException Demo ===");

        // Write some integers to a binary file
        try (DataOutputStream dos =
            new DataOutputStream(new FileOutputStream(filename))) {
            dos.writeInt(10);
            dos.writeInt(20);
            dos.writeInt(30);
        } catch (IOException e) {
            System.out.println("Write error: " + e.getMessage());
            return;
        }

        // Read back — EOFException when data is exhausted
        DataInputStream dis = null;
        try {
            dis = new DataInputStream(new FileInputStream(filename));
            while (true) {
                int value = dis.readInt(); // throws EOFException at end
```

```

        System.out.println("Read: " + value);
    }
} catch (EOFException e) {
    System.out.println("EOFException caught: end of file reached.");
} catch (IOException e) {
    System.out.println("IOException: " + e.getMessage());
} finally {
    try { if (dis != null) dis.close(); } catch (IOException ignored) {}
}
}

public static void main(String[] args) {
    demoClassNotFound();
    demoEOF();
}
}

```

Sample Input

(No user input required — program uses hardcoded class name and file data.)

Sample Output

```

=== ClassNotFoundException Demo ===
ClassNotFoundException caught!
Message : com.nonexistent.DatabaseDriver

=== EOFException Demo ===
Read: 10
Read: 20
Read: 30
EOFException caught: end of file reached.

```

Problem No: 07**Problem Name:**

Write a program in “JAVA” or “C” that will read an input.txt file containing n positive integers and calculate addition, subtraction, multiplication and division in a separate output.txt file.

Sample input: 5 5 9 8

Sample output:

Case-1: 10 0 25 1

Case-2: 17 1 72 1

Objectives:

1. To develop a C program that reads integer values from an input file.
2. To perform arithmetic operations such as addition, subtraction, multiplication, and division on pairs of numbers.
3. To write the calculated results into an output file in a structured format.

Theory:

File handling in C allows a program to read data from external files and store results in output files. It is useful for processing large datasets without manual input.

In this program:

- Data is read from input.txt
- Integers are stored in an array
- Each pair of numbers is processed using arithmetic operations:
 - Addition (+)
 - Subtraction (-)
 - Multiplication (*)
 - Division (/)

The results are written into output.txt in the format:

Case-1: a+b a-b a*b a/b

Important considerations:

- Input must contain an even number of integers
- Division by zero must be handled safely
- File operations use fopen(), fscanf(), fprintf(), and fclose()

This experiment helps to understand file processing, array manipulation, and error handling in C programming.

Algorithm:

1. Start
2. Open the input file input.txt in read mode.
 - If file opening fails, display error and terminate.
3. Open the output file output.txt in write mode.
 - If file opening fails, display error and terminate.
4. Read integers from input.txt.
 - Assume the number of integers n is even (pairs of numbers).
 - Alternatively, read until end of file.
5. For every pair of numbers (a, b) read:
 - Calculate:
 - Addition = a + b
 - Subtraction = a - b
 - Multiplication = a * b
 - Division = a / b (integer division; check for division by zero)

Source Code

```
#include <stdio.h>

int main() {
    FILE *fin = fopen("input.txt", "r");
    FILE *fout = fopen("output.txt", "w");

    if (!fin) { printf("Cannot open input.txt\n"); return 1; }
    if (!fout) { printf("Cannot open output.txt\n"); return 1; }

    int arr[100];
    int n = 0;

    /* Read all integers from file */
    while (fscanf(fin, "%d", &arr[n]) == 1) {
        n++;
    }
    fclose(fin);

    int caseNum = 1;
    for (int i = 0; i + 1 < n; i += 2) {
        int a = arr[i];
        int b = arr[i + 1];
        int add = a + b;
        int sub = a - b;
        int mul = a * b;
        int div = (b != 0) ? a / b : 0;

        fprintf(fout, "Case-%d: %d %d %d %d\n",
            caseNum, add, sub, mul, div);
        caseNum++;
    }

    fclose(fout);
    printf("Processing complete. Results written to output.txt\n");
    return 0;
}
```

Sample Input

Contents of input.txt:

5 5 9 8

Sample Output

Contents of output.txt:

Case-1: 10 0 25 1

Case-2: 17 1 72 1

Console:

Processing complete. Results written to output.txt

Problem No: 08

Problem Name:

Write down the ATM system specifications and report the various bugs.

Objectives:

To understand the functional and non-functional requirements of an ATM system.

1. To identify the hardware and software components used in ATM operations.
2. To analyze common bugs and issues that may occur in ATM systems and affect reliability, security, and user experience.

Theory:

An Automated Teller Machine (ATM) is a computerized banking system that allows customers to perform financial transactions without visiting a bank branch.

ATM System Specifications

2.1 Functional Requirements (FR)

- **Card Validation:** The system must accept bank cards and read serialized information.
- **Authentication:** The system must request and verify a 4-digit PIN against the bank server.
- **Transactions:** The system must facilitate cash withdrawals, cash deposits, and balance inquiries.
- **Receipt Generation:** The system must generate a physical receipt after successful transactions.

2.2 Non-Functional Requirements (NFR)

- **Availability:** The ATM must provide 24/7 times service.
- **Performance:** Transaction processing time should be less than 3seconds under normal load and less than 5 seconds under heavy load.
- **Security:** Non-volatile memory is required to ensure data integrity.

3. Bug Report

The following bugs were identified during testing:

Bug ID	Description	Severity	Priority
ATM-01	Card remains inside the slot after a transaction fails due to network error, requiring manual reset.	High	High
ATM-02	System allows multiple withdrawal attempts exceeding the daily limit (e.g., more than \(\$500\)) if made rapidly.	Critical	High
ATM-03	PIN field allows entry of more than 4 characters, leading to screen overflow.	Medium	Low
ATM-04	The "Balance Inquiry" option displays the wrong balance if a, deposit has just been made.	High	High
ATM-05	Printing receipt fails, but the transaction still processes, resulting in a silent failure.	Medium	Medium

Problem No: 09**Problem Name:**

Explain the role of software engineering in Biomedical Engineering and in the fields of Artificial Intelligence and Robotics.

Objectives:

1. To understand the importance of software engineering in Biomedical Engineering applications.
2. To explain the role of software engineering in Artificial Intelligence (AI) and Robotics systems.
3. To analyze how software solutions improve automation, healthcare systems, and intelligent technologies.

Theory:

Software engineering is the process of designing, developing, testing, and maintaining software systems in an organized way. Nowadays, software engineering is used in almost every modern technology field. It has a very important role in Biomedical Engineering, Artificial Intelligence (AI), and Robotics because these fields depend heavily on software for operation, analysis, and control.

Role of Software Engineering in Biomedical Engineering

Biomedical Engineering combines medical science with engineering technology to improve healthcare systems and medical treatment. Software engineering helps this field in many ways.

1. Medical Software Development

Different types of software are used in hospitals, clinics, and diagnostic centers. Software engineers develop applications for:

- Electronic Health Records (EHR)
- Medical image analysis
- Patient monitoring systems
- Hospital management systems

These software systems help doctors and nurses store patient information, monitor health conditions, and make faster decisions.

2. Control of Medical Devices

Many biomedical devices work using embedded software. Devices such as pacemakers, infusion pumps, ECG machines, and prosthetic limbs require software to operate properly.

Software engineering helps in writing reliable programs that control these devices accurately and safely.

3. Biomedical Signal Processing

Signals like ECG, EEG, and EMG contain important medical information. Software is used to process these signals by:

- Removing unwanted noise
- Detecting abnormalities
- Analyzing patient conditions
- Assisting doctors in diagnosis

4. Safety and Reliability

Biomedical software must be highly accurate because human life depends on it. Software engineers perform testing, debugging, and validation to ensure that the system works correctly without failure.

5. Simulation and Research

Software tools such as MATLAB and Simulink are used for simulation and research in biomedical systems. Researchers can test models and medical devices virtually before practical implementation.

Role of Software Engineering in Artificial Intelligence and Robotics

Artificial Intelligence and Robotics are modern technologies that also depend greatly on software engineering.

1. Development of AI Algorithms

AI systems use software algorithms to perform intelligent tasks such as:

- Speech recognition
- Image processing
- Decision making
- Machine learning

Software engineers design and improve these algorithms so that machines can learn and solve problems efficiently.

2. Robot Control Systems

Robots need software to control their movement, sensors, and actions. Software engineering helps robots perform tasks automatically and accurately.

For example, robots in industries, hospitals, and research labs use software to move, detect objects, and respond to commands.

3. Simulation and Testing

Before using robots or AI systems in real environments, they are tested using simulation software. This helps engineers identify errors and improve system performance.

4. Human-Robot Interaction

Software engineering also helps create communication systems between humans and robots. Robots can interact with humans through voice commands, touch systems, or visual recognition.

5. Security and Safety

AI and robotic systems must operate safely. Software engineers develop secure and reliable systems to avoid malfunction and reduce risks during operation.

Problem No: 10

Problem Name:

Study the various phases of the Waterfall Model. Which phase is the most dominant one?

Objectives:

1. To study the different phases of the Waterfall Software Development Model.
2. To understand the purpose and activities of each phase in the software development process.
3. To identify the most important and dominant phase of the Waterfall Model.

Theory:

The Waterfall Model is one of the oldest and simplest software development models used in Software Engineering. It follows a linear and sequential approach, where one phase must be completed before moving to the next phase. Each stage depends on the completion of the previous stage.

The Waterfall Model is easy to understand and widely used for projects where requirements are clearly defined from the beginning.

Phases of the Waterfall Model

1. Requirement Analysis

This is the first phase of the Waterfall Model. In this phase, all system requirements are collected from the customer or user.

The development team studies:

- What the customer wants
- System functionality
- Input and output requirements
- Hardware and software requirements

After collecting all information, a Software Requirement Specification (SRS) document is prepared.

This phase is very important because the entire project depends on correct requirements.

2. System Design

In this phase, the overall system design is prepared based on the requirements gathered earlier.

The design includes:

- System architecture
- Database design
- User interface design
- Module structure
- Data flow design

This phase provides a blueprint for developers before coding starts.

3. Implementation (Coding)

In the implementation phase, programmers write the actual source code according to the system design.

Different modules of the software are developed separately using programming languages such as:

- C
- Java
- Python
- C++

This phase converts the design into a working software system.

4. Integration and Testing

After coding is completed, all modules are integrated together and tested carefully.

Testing is performed to identify and remove errors or bugs. Different types of testing include:

- Unit Testing
- Integration Testing
- System Testing
- Acceptance Testing

The main goal of this phase is to ensure that the software works correctly according to requirements.

5. Deployment

In this phase, the software is delivered and installed in the customer environment for actual use.

Users start using the system in real-world conditions.

6. Maintenance

Maintenance is the final phase of the Waterfall Model. After deployment, developers continue supporting the software by:

- Fixing bugs
- Updating features
- Improving performance
- Handling security issues

This phase may continue for a long period after software release.

Most Dominant Phase of the Waterfall Model

The Requirement Analysis phase is considered the most dominant and important phase of the Waterfall Model.

Reasons:

- It forms the foundation of the entire project.
- All other phases depend on correct requirements.
- Wrong or incomplete requirements lead to software failure.
- Errors in this phase are expensive and difficult to fix later.

If requirements are properly collected and documented, the development process becomes smoother and more successful.

Some experts also consider the Maintenance phase dominant because it consumes a large amount of time and cost in real-life projects. However, from a development perspective, Requirement Analysis is usually regarded as the most critical phase.

Problem No: 11

Problem Name:

Using COCOMO model estimate effort for specific problem in industrial domain.

Objectives:

1. To understand the basic concept of the COCOMO model used in software cost estimation.
2. To estimate software development effort and schedule for an industrial project using the model.
3. To analyze how project size affects cost and development time in real-world software systems.

Theory:

COCOMO (Constructive Cost Model) is a software estimation model developed by Barry Boehm. It is used to estimate the effort, cost, and development time required for a software project based on its size.

In software engineering, estimation is very important because it helps in planning resources, time, and budget before starting a project.

COCOMO mainly depends on the size of the software, which is measured in KLOC (Kilo Lines of Code).

There are three types of COCOMO models:

1. Basic COCOMO Model

This model estimates effort only based on program size.

The main formula is:

- $\text{Effort (Person-Months)} = a \times (\text{KLOC})^b$

Where:

- KLOC = Thousand Lines of Code
- a, b = constants depending on project type

Project types include:

- Organic (simple projects, small teams)
- Semi-detached (medium complexity projects)
- Embedded (complex, real-time, hardware-related systems)

2. Intermediate COCOMO Model

This model includes cost drivers such as:

- Product reliability
- Team experience
- Hardware constraints
- Project complexity

It gives more realistic results compared to Basic COCOMO.

3. Detailed COCOMO Model

This is the most advanced model. It considers:

- Each phase of software development

- Detailed cost drivers
- Phase-wise effort estimation

Industrial Domain Example:

Let us consider an industrial project such as an **Industrial Process Monitoring System** used in a factory to monitor temperature, pressure, and machine performance in real time.

Assumptions:

- Estimated size = 50 KLOC
- Project type = Embedded system (since it is real-time and hardware dependent)

For Embedded systems:

- $a = 3.6$
- $b = 1.20$

Effort Estimation:

Using COCOMO formula:

$$\text{Effort} = 3.6 \times (50)^{1.20}$$

$$(50)^{1.20} \approx 109.4$$

So,

$$\text{Effort} \approx 3.6 \times 109.4 \approx 393.8 \text{ Person-Months}$$

Development Time Estimation:

Development time formula:

- $\text{Time} = c \times (\text{Effort})^d$

For Embedded systems:

- $c = 2.5$
- $d = 0.32$

$$\text{Time} \approx 2.5 \times (393.8)^{0.32}$$

$$\text{Time} \approx 2.5 \times 7.2$$

$$\text{Time} \approx 18 \text{ months}$$

Problem No: 12

Problem Name:

Identify the reasons behind the software crisis and explain the possible solutions for the following scenario:

Case 1: Air ticket reservation software was delivered to the customer and was installed in an airport 12.00 AM (midnight) as per the plan. The system worked quite fine till the next day 12.00 PM (noon). The system crashed at 12.00 PM, and the airport authorities could not continue using the software for ticket reservation till 5.00 PM. It took 5 hours to fix the defect in the software.

Case 2: Software for financial systems was delivered to the customer. The customer informed the development team about a malfunction in the system. As the software was huge and complex, the development team could not identify the defect in the software.

Objectives:

1. To understand the causes behind the software crisis in real-world systems.
2. To analyze common failures in software projects after deployment.
3. To suggest practical solutions for improving software quality, reliability, and maintainability in complex systems.

Theory:

The term software crisis refers to the difficulties faced in developing large and complex software systems. It includes problems like project delays, high cost, system failures, poor quality, and difficulty in maintenance. As software systems become more complex, these issues become more common if proper engineering practices are not followed.

Reasons Behind Software Crisis

1. Poor Requirement Analysis

One of the main reasons for software failure is unclear or incomplete requirement gathering. If requirements are not properly understood at the beginning, the final system may not match user needs.

2. Lack of Proper Design

Without a well-structured design, software becomes unorganized and difficult to manage. This leads to errors and poor system performance.

3. Inadequate Testing

Many software systems are not tested under real-world conditions. As a result, hidden bugs appear after deployment.

4. Increasing System Complexity

Modern software systems are large and interconnected. Managing such complexity without proper planning often leads to failure.

5. Poor Documentation

If system design and code are not properly documented, future maintenance becomes very difficult.

6. Time and Cost Pressure

Developers are often forced to deliver software within short deadlines, which reduces software quality.

7. Lack of Maintenance Strategy

Many systems fail because they do not have a proper plan for updates, bug fixes, and long-term maintenance.

Case Analysis and Solutions:

Case 1: Air Ticket Reservation System Failure

Problem Description:

The system worked properly after installation but suddenly crashed after 12 hours of operation, causing a 5-hour service disruption.

Possible Causes:

- Time handling error (AM/PM or date rollover issue)
- Memory leak causing system overload
- Insufficient load testing before deployment
- Poor error handling and monitoring system
- Lack of stress testing under real airport conditions

Solutions:

- Perform strong boundary testing (especially time-based scenarios)
- Conduct load and stress testing before deployment
- Implement real-time monitoring and alert systems
- Use backup or failover systems for continuous service
- Improve logging for faster debugging and recovery

Case 2: Financial System Bug Detection Failure

Problem Description:

A large financial system had a malfunction, but the development team could not identify the defect due to system complexity.

Possible Causes:

- Poor system modularity (monolithic design)
- Lack of proper logging and debugging tools
- Incomplete documentation
- High code complexity and tight coupling between modules
- Insufficient unit testing

Solutions:

- Use modular or microservice-based architecture
- Improve logging and tracing mechanisms
- Apply test-driven development (TDD)
- Maintain proper documentation and system diagrams
- Use debugging tools and automated testing frameworks
- Conduct regular code refactor